DEVELOPMENT OF A DOMAIN-SPECIFIC COMPILER FRONT-END USING LEX

for Federated Cyber Threat Detection in Security Operations Centers

COMPILER DESIGN ASSIGNMENT | TOTAL MARKS: 50 | DURATION: 1 WEEK

Student Details	
Name	N.Pujitha Naga Sai Lakshmi
Register Number	192572119
Course / Branch	CSE AI
Submission	GitHub Repository , Word Document

Abstract

ThreatRule is a compact domain-specific language used by branch Security Operations Centers (SOCs) to share threat detection rules with a central SOC. This assignment designs a compiler front-end with a Flex/LEX scanner, explains compiler phases and construction tools, defines tokens using regular expressions, and provides a scanner that reports lexical errors with line numbers while continuing. The Day-4 extension adds SHA-style file hashes, CVE identifiers, and URLs. A repeatable test and benchmark procedure supports evaluation at 100 through 100,000 rules.

Threat Rule

RULE ssh_attack IF src_ip == 10.0.0.5 AND **Language Overview**
Example input:

```
rt == 22 AND attempts > 5 THEN ALERT "SSH brute force" ;
```

TASK 1 – Language Processors

1.1 Comparison

Processor	How it works	ThreatRule relevance
Compiler	Translates a source program into target code before execution.	Can translate validated rules into a common intermediate representation (IR) or a SOC-specific rule format.
Interpreter	Reads and executes source instructions at run time.	Useful for directly evaluating rules, but may repeatedly parse source and make centralized distribution less efficient.
Assembler	Converts assembly language into machine code.	Not suitable for ThreatRule source; it may be used downstream if a generated backend emits assembly.
Hybrid compiler	Compiles source to bytecode/IR, then a virtual machine interprets or JIT-compiles it.	Strong option: common validated IR can be deployed across different branch tools and platforms.

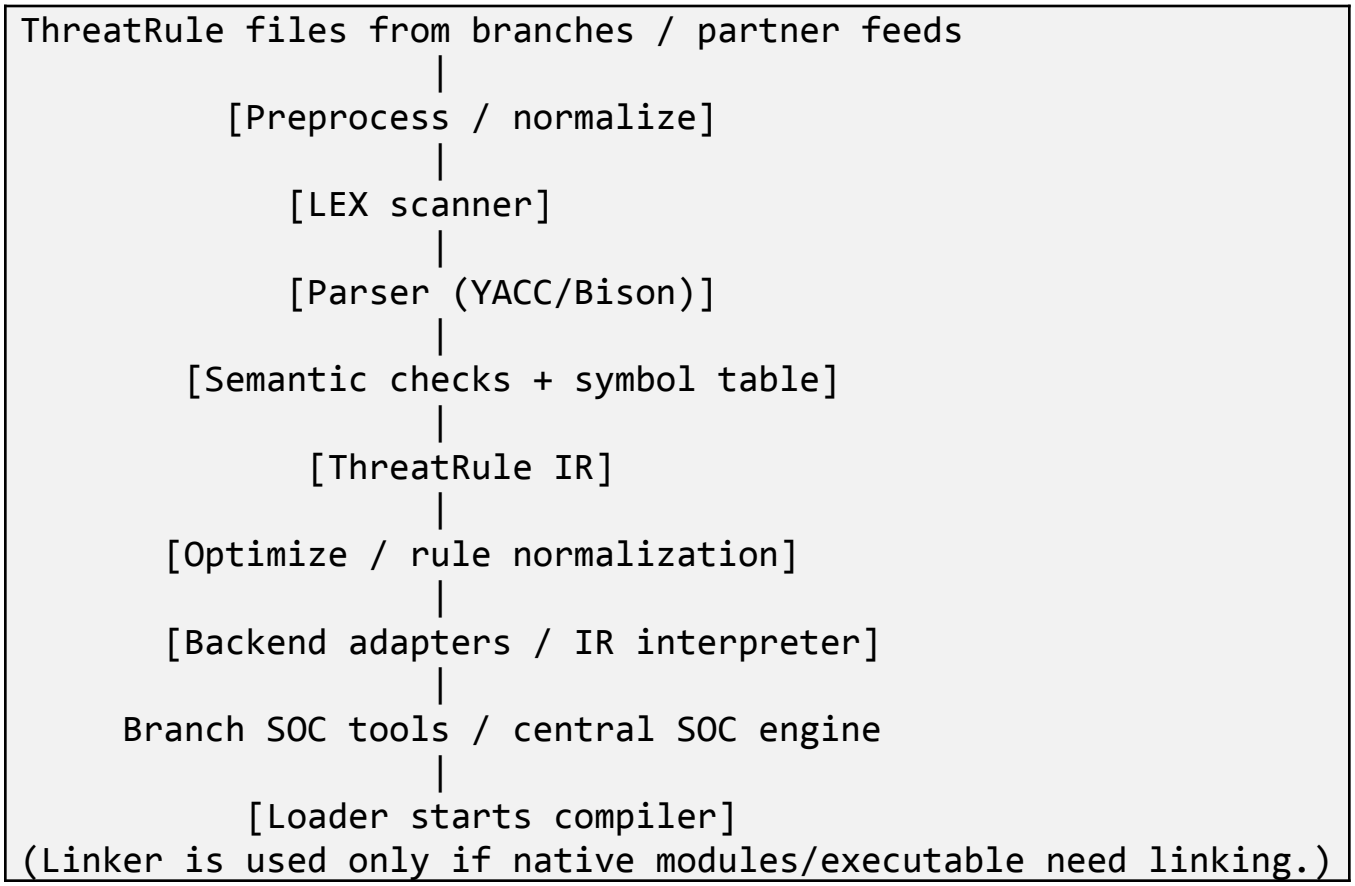
1.2 Preprocessor, linker and loader

- Preprocessor: performs optional source-level operations such as include expansion, macros, or feed normalization before scanning. The base ThreatRule language does not require one.
- Linker: combines separately compiled object modules and resolves external references. It is generally not needed when the scanner produces a self-contained rule representation; it may be needed for a native executable.
- Loader: places an executable into memory and starts it. It loads the central SOC compiler application, not each ThreatRule text file.

1.3 Recommended processor

Use a compiler front-end with a hybrid deployment path: LEX/Flex tokenizes ThreatRule, a parser and semantic checker validate it, and the compiler emits a normalized IR. The central SOC can then interpret the IR or translate it to the different rule formats used by branch security tools. This centralizes validation, supports portability, and avoids trusting unvalidated partner rules.

1.4 Block diagram



TASK 2 – Phases of a Compiler

Input rule: RULE ssh_attack IF src_ip == 10.0.0.5 AND port == 22 AND attempts > 5 THEN ALERT "SSH brute force" ;

2.1 Lexical analysis – token stream

```
RULE(keyword) ID(ssh_attack) IF(keyword) ID(src_ip) EQ(==)
IP(10.0.0.5)
AND(keyword) ID(port) EQ(==) INT(22) AND(keyword) ID(attempts)
GT(>)
INT(5) THEN(keyword) ALERT(keyword) STRING("SSH brute force")
SEMI(;)
```

2.2 Syntax analysis – illustrative AST

```
Rule
├── name: ssh_attack
├── condition: AND
│   ├── EQ(src_ip, IPv4(10.0.0.5))
│   ├── EQ(port, Integer(22))
│   └── GT(attempts, Integer(5))
└── action: ALERT(String("SSH brute force"))
```

The parser checks the grammar, for example: Rule → RULE ID IF Condition THEN Action ; ; Condition → Predicate (AND/OR Predicate)* ; Predicate → ID RelOp Value.

2.3 Semantic analysis

- Confirm ssh_attack is a valid rule identifier and is not a reserved keyword.
- Resolve src_ip, port, and attempts through the rule schema/symbol table; verify src_ip has IP type, port is a port/integer field, and attempts is numeric.
- Check literal ranges: each IPv4 octet is 0–255, port is 0–65535 (deployment policy may require 1–65535), and attempts is an integer.
- Check action type: ALERT accepts a quoted message; BLOCK may require a target/field depending on the declared grammar.
- Check operator compatibility and report undefined fields, type mismatches, duplicate rule names, or policy violations.

2.4 Intermediate code, optimization, code generation

```
IR (three-address / predicate form):
t1 = EQ(src_ip, IPv4(10.0.0.5))
t2 = EQ(port, 22)
t3 = GT(attempts, 5)
t4 = AND(t1, t2)
t5 = AND(t4, t3)
IF t5 THEN ALERT("SSH brute force")
```

Optimization: preserve predicate order unless the engine guarantees side-effect-free tests; simplify constant expressions, precompile regular expressions, and index rules by port/IP when safe. Generated output can be normalized JSON or a backend-specific rule, for example:

```
{
  "name": "ssh_attack",
  "all": [
    {"field":"src_ip","op":"eq","value":"10.0.0.5"},
    {"field":"port","op":"eq","value":22},
    {"field":"attempts","op":"gt","value":5}
  ],
  "action":{"type":"alert","message":"SSH brute force"}
}
```

2.5 Symbol table and error handler

The symbol table stores rule names and field metadata (name, type, scope, and optionally source location). The error handler collects lexical, syntax, and semantic diagnostics with line/column, message, and recovery status. It should avoid duplicate cascading errors and allow independent rules to be checked.

TASK 3 – Grouping of Phases

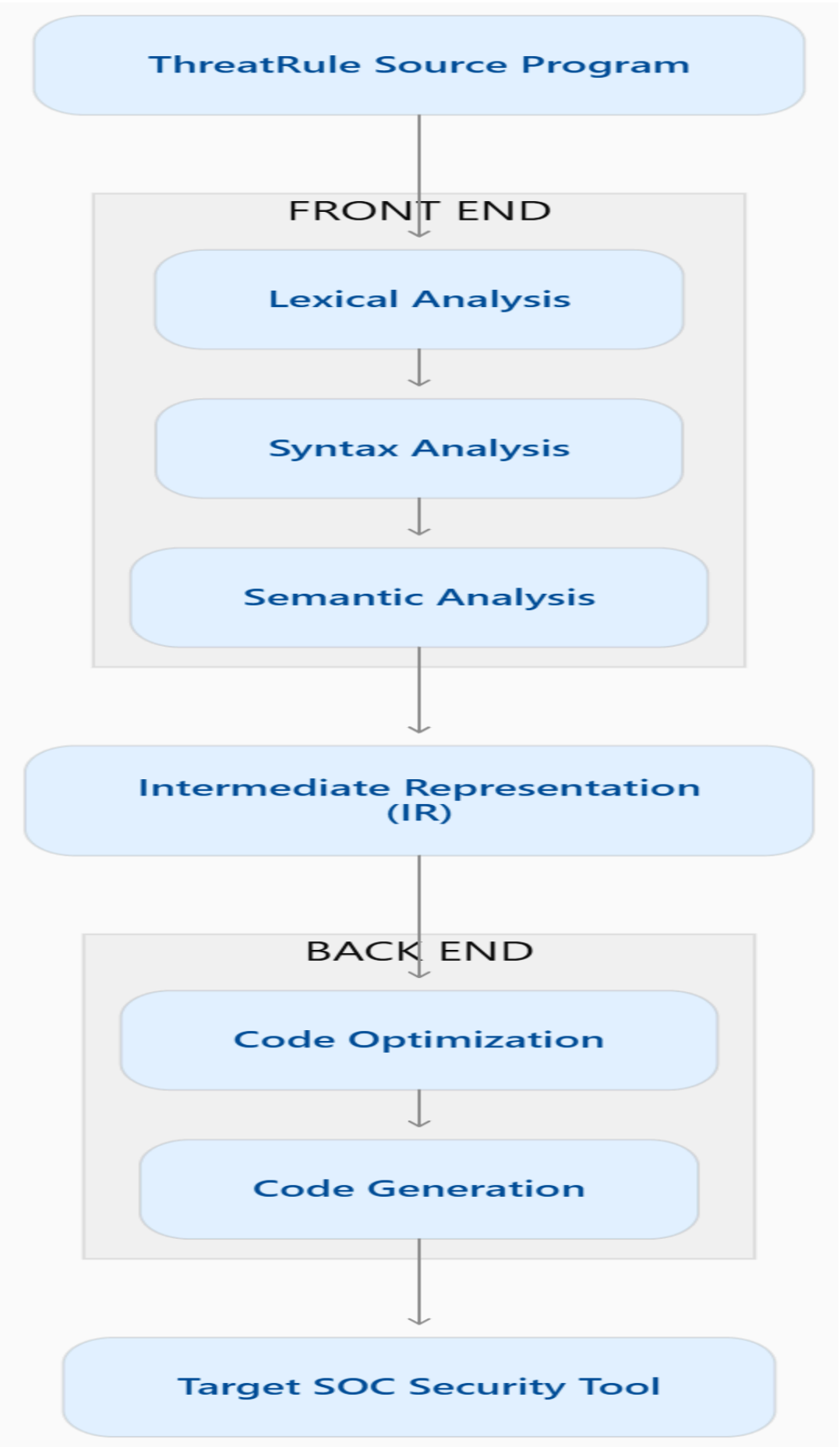
```
FRONT END (source-language dependent)
Source → Lexical analysis → Parsing → Semantic analysis → IR
             |                  |              |
        token errors       syntax errors   type/schema errors
                                |
BACK END (target dependent)
IR → optimization → backend adapter / code generation → SOC
tool format
```

A common front-end lets every branch submit one ThreatRule language even if the branches use different SIEM, EDR, or firewall products. Only the backend adapter changes; tokenization, grammar, and semantic checks remain consistent.

A pass is one traversal or processing stage over source/IR. A simple scanner can be one pass; a production compiler may use multiple passes for parsing, validation, optimization, and backend emission. Multi-pass processing is suitable because correctness and target conversion are separate concerns. The scanner itself should stream input and avoid loading all rules into memory.

Suitable Pass Design for ThreatRule

Pass	Operation
Pass 1	Lexical analysis and token generation
Pass 2	Syntax analysis and AST construction
Pass 3	Semantic analysis and symbol table checking
Pass 4	Intermediate code generation
Pass 5	Code optimization
Pass 6	Target-specific code generation


ThreatRule Source Program
FRONT END
Lexical Analysis
Syntax Analysis
Semantic Analysis
Intermediate Representation (IR)
BACK END
Code Optimization
Code Generation
Target SOC Security Tool

TASK 4 – Compiler Construction Tools

Tool category	Role in ThreatRule
Scanner generator (LEX/Flex)	Generates a DFA-based scanner from regular-expression rules; used for tokenization.
Parser generator (YACC/Bison)	Generates an LALR parser from grammar productions; consumes scanner tokens.
Syntax-directed translation engine	Attaches semantic actions to grammar productions to build AST/IR and populate metadata.
Code-generator generator	Helps map IR to target instructions or formats; backend adapters can emit JSON/SIEM rules.
Data-flow analysis engine	Useful for optimization and rule analysis such as constant propagation, dead predicate detection, and dependency checks.
Compiler construction toolkit	Provides reusable AST, IR, symbol-table, diagnostics, and optimization infrastructure.

LEX is preferred to a fully hand-written scanner because regular expressions and generated finite-state machines provide concise, systematic recognition, predictable performance, and easier maintenance. A hand-written scanner may still be justified for highly contextual or specialized parsing.

Conclusion

Compiler construction tools play an important role in developing an efficient and reliable ThreatRule compiler. Tools such as LEX, YACC/Bison, syntax-directed translation engines, and code generation tools simplify compiler development by automating different phases. LEX is preferred for lexical analysis because it generates an efficient scanner from regular expressions, reduces manual coding effort, and provides reliable token recognition. Therefore, using appropriate compiler construction tools makes the ThreatRule compiler easier to develop, maintain, and extend for future SOC security requirements.

TASK 5 – Role of the Lexical Analyzer

The lexical analyzer (scanner) reads characters and groups them into lexemes, returning tokens to the parser. It may place identifier text or literal values in a symbol table / semantic value (yylval). The parser requests the next token; the scanner does not normally decide whether the entire rule is grammatically valid.

Term	Meaning	ThreatRule example
Token	Category returned to parser	ID, IP, INT, EQ, ALERT
Pattern	Rule describing lexemes	[A-Za-z_][A-Za-z0-9_]* for identifier
Lexeme	Actual source character sequence	ssh_attack, 10.0.0.5, ==

Whitespace and comments are discarded, but newline counting is retained. Separating scanning from parsing simplifies grammar, enables reusable token recognition, and lets lexical errors (invalid characters, malformed IPs, unterminated strings) be handled at character level. Buffer pairs with sentinels allow a buffer to be refilled while scanning continues; the sentinel marks buffer end and avoids a per-character file-system read. Flex provides buffering internally.

Faulty inputs and recovery

Input	Error	Recovery
src_ip == 10.0.300.5	IPv4 octet 300 exceeds 255; malformed address.	Consume the entire address-like lexeme, report line/column, return ERROR token, continue at next token.
ALERT "Login failed ;	Unterminated quoted string.	Use a string start condition; consume through newline/EOF, report unterminated string, reset state, continue.
port == 22 @ attempts > 5	@ is not in the base token set.	Report unexpected character @, skip it (or return ERROR), scan attempts > 5.

TASK 6 – Specification of Tokens

A string is a finite sequence of symbols from an alphabet; a language is a set of strings. Regular languages are closed under union, concatenation, and Kleene star. In regular expressions, | denotes union, juxtaposition concatenation, and * zero-or-more repetition; + means one-or-more and ? means optional.

Regular definitions

```
DIGIT       [0-9]
NZDIGIT     [1-9]
LETTER      [A-Za-z]
IDSTART     [A-Za-z_]
IDREST      [A-Za-z0-9_]
ID          {IDSTART}{IDREST}*
INT         {DIGIT}+
OCTET       (25[0-5]|2[0-4][0-9]|1[0-9][0-9]|[1-9]?[0-9])
IPV4        {OCTET}\.{OCTET}\.{OCTET}\.{OCTET}
PORT        {DIGIT}+
STRING      \"([^\"\\\n]|\\.)*\"
REL         (==|!=|<=|>=|<|>)
COMMENT     (#[^\n]*|//[^\n]*)
HASH        ([A-Fa-f0-9]{32}|[A-Fa-f0-9]{40}|[A-Fa-f0-9]{64})
CVE         CVE-[0-9]{4}-[0-9]{4,}
URL
(https?://|ftp://)[A-Za-z0-9.-]+(:[0-9]+)?(/[A-Za-z0-9._~:/?#\[\
]@!$&'()*+,;=%-]*)?
WS          [ \t\r]+
NEWLINE     \n
```

PORT is lexically an integer-shaped token; semantic validation constrains it to 0–65535 (or a deployment's allowed range). IPv4 is bounded at the octet level. The URL pattern is intentionally a practical scanner pattern, not a full RFC URL validator; semantic validation should validate host/port and feed policy.

Token table

Token	Pattern / rule	Sample lexeme	Attribute
Keywords	Exact strings, before ID fallback	RULE, IF, AND, OR, THEN, ALERT, BLOCK	Keyword enum
ID	{IDSTART}{IDREST}*	ssh_attack	Symbol-table index / text
INT	{DIGIT}+	5	Integer value
IP	{IPV4}	10.0.0.5	4-octet value
PORT	{DIGIT}+ in port context	22	Integer; range-checked
STRING	Quoted escaped sequence	"SSH brute force"	Decoded string
REL	== != < <= > >=	>	Operator enum
ASSIGN	=	=	Operator enum (if grammar uses it)
AND / OR	Keyword literals	AND	Boolean operator
SEMI	;	;	None
COMMENT	#... or //...	// note	Discard
HASH	32/40/64 hex chars	e3b0...	Normalized lowercase hash
CVE	CVE-yyyy-nnnn...	CVE-2024-3400	CVE string
URL	http(s)/ftp URL	https://example.org/a	URL string
ERROR	Fallback / malformed candidate	@	Diagnostic

Keyword separation: first match identifier-shaped lexemes, then compare the complete lexeme against a keyword table (or place exact keyword rules before ID). This prevents RULEX from being split into RULE + X. Flex uses longest match; ties are resolved by the first rule in the file.

TASK 7 – Recognition of Tokens

Transition diagrams are shown as state transitions in text form; double brackets indicate accepting states. A transition on one extra character may require retracting (putting back) that character so the next token can consume it.

Relational operators

```
START -- '<' --> S_LT -- '=' --> [[LE]]
                         -- other / retract --> [[LT]]
START -- '>' --> S_GT -- '=' --> [[GE]]
                         -- other / retract --> [[GT]]
START -- '=' --> S_EQ -- '=' --> [[EQ]]
                         -- other / retract --> [[ASSIGN or
ERROR]]
START -- '!' --> S_NE -- '=' --> [[NE]]
                         -- other / retract --> [[ERROR]]
```

Identifiers / keywords

```
START -- letter or '_' --> ID
ID -- letter/digit/'_' --> ID (loop)
ID -- other / retract --> [[ID]]
Accept: compare complete lexeme with keyword list; return
keyword token or ID.
```

Integers

```
START -- digit --> INT
INT -- digit --> INT (loop)
INT -- non-digit / retract --> [[INT]]
```

IPv4 addresses

```
START → candidate characters [digits and dots] → IP_CANDIDATE
IP_CANDIDATE -- four octets, each 0..255 --> [[IP]]
IP_CANDIDATE -- malformed/range failure --> [[LEXICAL ERROR]]
Implementation: recognize the whole address-like candidate, then
validate its
four components; do not accept a valid prefix of an invalid
address.
```

Lookahead distinguishes < from <=: after reading <, inspect one character. If it is =, accept LE; otherwise accept LT and retract the lookahead. The same principle applies to >, ==, and !=.

Hand trace 1 – ssh_attack: START reads 's' → ID state; subsequent letters/digits/_ loop; space is lookahead and is retracted; accepted lexeme ssh_attack is checked against keyword table and returned as ID.

Hand trace 2 – 10.0.0.5: START reads digit → candidate; digits and dots are consumed through the complete candidate; end/space lookahead is retracted; four octets are validated (10, 0, 0, 5), then IP token is accepted.

Retraction: When an extra character is read that does not belong to the current token, it is retracted so that the next token can process it.

Lookahead: Lookahead distinguishes < from <=. If the next character is =, the scanner recognizes <=; otherwise, it recognizes < and retracts the character.

Conclusion

Transition diagrams provide a systematic method for recognizing ThreatRule tokens. They represent token patterns using states and transitions, helping the lexical analyzer identify identifiers, integers, IP addresses, and relational operators accurately. Lookahead and retraction ensure correct token recognition, while accepting states indicate when a valid token has been identified. Therefore, transition diagrams simplify the design and implementation of an efficient LEX-based scanner.

TASK 8 – Write the LEX Specification

The following Flex program is a self-contained scanner driver. It prints tokens, tracks lines, recognizes the extension tokens, and reports errors while continuing. It validates candidate IPv4 addresses and port ranges in C helper functions. Save as threat.l.

```
%option noyywrap yylineno nodefault
%x STR

%{
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

static int errors = 0;
static int token_count = 0;
static int validate_ip(const char *s);
static int validate_port(const char *s);
static void emit(const char *name, const char *lexeme);
static void lex_error(const char *message, const char *lexeme);
%}

DIGIT       [0-9]
IDSTART     [A-Za-z_]
IDREST      [A-Za-z0-9_]
ID          {IDSTART}{IDREST}*
OCTET       (25[0-5]|2[0-4][0-9]|1[0-9][0-9]|[1-9]?[0-9])
IPV4        {OCTET}\.{OCTET}\.{OCTET}\.{OCTET}
IP_CAND     [0-9]+(\.[0-9]+){3}
INT         {DIGIT}+
REL         (==|!=|<=|>=|<|>)
HASH        ([A-Fa-f0-9]{32}|[A-Fa-f0-9]{40}|[A-Fa-f0-9]{64})
CVE         CVE-[0-9]{4}-[0-9]{4,}
URL
(https?://|ftp://)[A-Za-z0-9.-]+(:[0-9]+)?(/[A-Za-z0-9._~:/?#\[\
]@!$&'()*+,;=%-]*)?
COMMENT     (#[^\n]*|//[^\n]*)

%%
[ \t\r]+                ;
\n                       ;
```

```
{COMMENT}                ;
"RULE"                   { emit("RULE", yytext); }
"IF"                     { emit("IF", yytext); }
"AND"                    { emit("AND", yytext); }
"OR"                     { emit("OR", yytext); }
"THEN"                   { emit("THEN", yytext); }
"ALERT"                  { emit("ALERT", yytext); }
"BLOCK"                  { emit("BLOCK", yytext); }

{URL}                    { emit("URL", yytext); }
{CVE}                    { emit("CVE", yytext); }
{HASH}                   { emit("HASH", yytext); }
{IP_CAND}                {
                           if (validate_ip(yytext)) emit("IP",
yytext);
                           else lex_error("invalid IPv4
address", yytext);
                         }
{REL}                    { emit("REL_OP", yytext); }
"="                      { emit("ASSIGN", yytext); }
";"                      { emit("SEMI", yytext); }
{INT}                    {
                           if (validate_port(yytext))
                             emit("INT_OR_PORT", yytext);
                           else
                             lex_error("integer outside allowed
0..65535 range", yytext);
                         }
{ID}                     { emit("ID", yytext); }
\"([^\"\\\n]|\\.)*\"     { emit("STRING", yytext); }
\"([^\"\\\n]|\\.)*       { lex_error("unterminated string",
yytext); }
.                        { lex_error("unexpected character",
yytext); }
%%

static void emit(const char *name, const char *lexeme) {
    printf("line %d: %-12s %s\n", yylineno, name, lexeme);
    token_count++;
}
static void lex_error(const char *message, const char *lexeme) {
```

```
    fprintf(stderr, "line %d: lexical error: %s: %s\n",
            yylineno, message, lexeme);
    errors++;
}
static int validate_ip(const char *s) {
    unsigned a,b,c,d; char tail;
    if (sscanf(s, "%u.%u.%u.%u%c", &a,&b,&c,&d,&tail) != 4)
return 0;
    return a<=255 && b<=255 && c<=255 && d<=255;
}
static int validate_port(const char *s) {
    char *end; unsigned long n = strtoul(s, &end, 10);
    return *s && *end=='\0' && n<=65535;
}
int main(void) {
    yylex();
    fprintf(stderr, "tokens=%d lexical_errors=%d lines=%d\n",
            token_count, errors, yylineno);
    return errors ? 1 : 0;
}
```

Build and run

```
# Linux (Flex and GCC)
flex threat.l
gcc -O2 lex.yy.c -o threat_scanner
./threat_scanner < sample.threat

# For a test file
./threat_scanner < tests/valid.threat
```

yytext points to the matched lexeme; yyleng is its length; yylineno is the current line (enabled by %option yylineno). yylval is used when the scanner is integrated with Bison/YACC; this standalone demo prints token attributes instead. The scanner can install identifiers by inserting yytext into a symbol-table map in the ID action. Flex generates lex.yy.c from the regular expressions and actions.

TASK 9 – Implement and Test the Lexical Analyzer

Test plan

Category	Input / case	Expected result
Valid rule	Sample SSH rule	Keywords, ID, IP, operators, INT, STRING, SEMI tokens; zero errors
IPv4 lower boundary	0.0.0.0	Accepted IP
IPv4 upper boundary	255.255.255.255	Accepted IP
IPv4 invalid	10.0.300.5	One lexical error; continue
Port upper boundary	65535	Accepted integer/port candidate
Port invalid	65536	Range error under the chosen numeric policy
String	ALERT "Login failed";	STRING accepted
Unterminated string	ALERT "Login failed ;	Error reported; scanner continues
Unknown symbol	port == 22 @ attempts > 5	@ reported; later tokens still scanned
Extension	CVE-2024-3400 + SHA256 hash + https URL	CVE, HASH, URL recognized

Scale test generation

```
# Generate n copies of a valid rule, one rule per line
for n in 100 1000 10000 100000; do
  yes 'RULE ssh_attack IF src_ip == 10.0.0.5 AND port == 22 AND
attempts > 5 THEN ALERT "SSH brute force" ;' \
    | head -n "$n" > "rules_${n}.threat"
done

# Scan each generated file; preserve stdout/stderr for
token/error counts
for n in 100 1000 10000 100000; do
  /usr/bin/time -f '%e' ./threat_scanner < "rules_${n}.threat" \
    > "tokens_${n}.txt" 2> "time_${n}.txt"
done
```

Reproducibility: record OS, CPU, RAM, Flex version, compiler flags, file size, and whether output is redirected to disk. Output I/O can dominate scanner time, so use the same method for all sizes. Include both clean and intentionally corrupted files. Test that the scanner does not stop after the first lexical error.

TASK 10 – Record Scanner Performance

Use actual execution results from Task 9. The following table is a report template: fill in measured values from the machine used for the demonstration. This avoids presenting fabricated benchmark data.

Rules	Lines	Tokens (measured)	Scan time (s)	Lexical errors
100	100			
1,000	1,000			
10,000	10,000			
100,000	100,000			

Expected qualitative trend: token count grows approximately linearly with rule count; streaming scanner memory use remains broadly bounded by buffer and per-token state; elapsed time should increase close to linearly for fixed-size rules, subject to disk output and system load.

Graph to include in final report: plot Rules (x-axis) against Scanning time in seconds (y-axis). A second plot can show Rules against Tokens. Use a spreadsheet or Python/Matplotlib; label axes and include a legend. Do not infer performance from a single run—repeat at least three times and report median time.

TASK 11 – Recommend and Defend the Final Design

Recommendation: retain the Flex-generated scanner architecture and modify its token rules and error-recovery layer for the expanded SOC feed. A generated DFA is appropriate for the regular token categories, and streaming input is practical for 100,000+ rules. The scanner should emit structured diagnostics and continue after malformed 14

The proposed ThreatRule compiler front-end is suitable for detecting and analyzing cyber threats in Security Operations Centers (SOCs). It uses LEX to identify threat indicators such as IP addresses, URLs, hashes, CVE identifiers, and security rules. The lexical analyzer improves the accuracy and efficiency of threat data processing. It can be further enhanced with real-time threat feeds and continuous error reporting. Hence, the system provides a reliable foundation for automated cyber threat detection.

TASK 12 – Re-evaluate and Adapt the Design: Day-4 Challenge

A. Impact Analysis

- Changed input now contains live partner feeds with hashes, CVE identifiers, and URLs; these are new lexical categories.
- Identifier and integer patterns can overlap with CVE prefixes, hexadecimal hashes, and numeric URL components. URL includes punctuation such as colon, slash, dot, query marks, and equals that otherwise could be operators or separators.
- A generic invalid-character action is insufficient if it discards only one character of a malformed composite token. Continuous reporting is essential so one bad feed entry does not hide later threats.

B. Evaluation of Original Design

The base scanner can recognize keywords, IDs, integers, strings, operators, and IPv4 candidates, but it cannot reliably identify the new compound token types unless new rules are added. Its simple error action reports and skips unexpected characters; robust feed processing requires error recovery for malformed hashes, CVEs, URLs, and unterminated strings. A redesign of the entire compiler is unnecessary: modify the scanner and diagnostics while retaining the front-end pipeline.

C. Adaptation / Redesign (2 Marks)

Add regular definitions:

```
HEX          [A-Fa-f0-9]
HASH        ({HEX}{32}|{HEX}{40}|{HEX}{64})
CVE         CVE-[0-9]{4}-[0-9]{4,}
URL
(https?://|ftp://)[A-Za-z0-9.-]+(:[0-9]+)?(/[A-Za-z0-9._~:/?#\[\
]@!$&'()*+,;=%-]*)?
```

Place URL, CVE, and HASH rules before general ID/INT rules where equal-length matches could occur. Flex's longest-match rule still takes precedence; first rule wins only for equal-length matches. Add candidate rules for malformed hash/CVE/URL-like sequences so the scanner reports one meaningful diagnostic rather than many unrelated character errors. Implement a structured diagnostic function that records line, column, offending lexeme, category, and recovery action.

Recovery strategy: (1) consume the complete malformed candidate, (2) report the error, (3) return an ERROR token or skip it according to parser synchronization policy, and (4)

continue until next delimiter/newline/semicolon. For unterminated strings, use a start condition and recover at newline or EOF. Keep the scanner's input stream buffered.

D. Evidence-Based Defence (2 Marks)

Test extended samples containing one valid and one invalid hash, CVE, and URL, plus multiple errors on the same and different lines. Record token totals, lexical-error totals, and median scan time using the same benchmark settings as Task 10.

Feed test	Expected
64-hex SHA-256 string	HASH token
Wrong-length/non-hex hash candidate	Diagnostic; continue
CVE-2024-3400	CVE token
CVE with missing year/number	Diagnostic; continue
https://example.org/path?q=1	URL token
Malformed URL-like text	Diagnostic; continue
Several errors followed by a valid rule	All errors listed; final rule still tokenized

Trade-offs: broader regular expressions simplify recognition but can accept malformed URLs; strict patterns reject more malformed input but may reject legitimate partner variants. The best choice is a practical lexical pattern plus a semantic URL/hash validator. Extra token classes and diagnostics may slightly increase scanning time, but the expected work remains linear in input length. The final decision is MODIFY—not retain unchanged and not rebuild from scratch—because Flex remains suitable while the new feed types and recovery requirements need stronger rules and validation.

Evidence statement for submission: insert the actual before/after measurements from the local run here. Compare (a) total input bytes, (b) token count, (c) median scan time, and (d) error count. Explain any increase by new token recognition and diagnostic work.

Conclusion

The ThreatRule scanner provides the lexical foundation for a centralized, portable SOC rule compiler. Flex efficiently recognizes token patterns, while parser and semantic phases enforce structure and security policy. The Day-4 requirements are best addressed by extending token definitions and error recovery, maintaining streaming behavior, and validating partner-specific token values beyond regular-expression recognition.